Asian University for Women
Department of Computer Science
Chattogram, Bangladesh

CSCI 2002: Object Oriented Programming Project

Project Title:	*Employee Management System*
Group Number:	*[Group 9]*
Group Members:	*[Marwa Mossini (222200)], [Freshta Hassani & (222348)]*
Course Code:	*CSCI 2002 — Object Oriented Programming*
Submitted To:	*Nanziba Basnin*
Date of Submission:	*22/8/2026*

Acknowledgement

We would like to express our gratitude to our Professor, Nanziba Basnin and Ayon Das, for their guidance and support throughout the improvement of this project. We also thank the Department of Computer Science, Asian University for Women for providing the resources and environment to compete this work. Finaly, we thank our group members for their collaboration and effort throughout this project.

Table of Contents

1. Introduction
    1.1. Problem Statement
    1.2. Motivation & Objectives
2. System Analysis and Design
    2.1. UML Diagram(s)
    2.2. Class Description
    2.3. System Workflow
3. Implementation
    3.1. Tools and Technologies
    3.2. OOP Concepts Applied
    3.3. Important Implementation Details
4. Testing and Results
5. Challenges and Solutions
6. Conclusion

1. Introduction

1.1 Problem Statement

University HR and registrar offices have to monitor employees for the wide variety of work they perform, ranging from research, teaching, and other student-engaging services to administrative work and health-care staff — and pay them under different rules. This process is slow, prone to inaccuracies, and cumbersome on paper or with a single generic spreadsheet, and is difficult to answer basic questions like "what is our total payroll this month" or "how many staff work in health care." Automating this process is easy for Asian University for Women with the University Employee Management System. It divides workers into groups called departments, each having a specific pay rule in one of four categories (Academic, Student Services, Administration, Health Care), automatically calculates the salary for each department, and saves all the information on disk to protect it from losing data between sessions. The model was used to create both a console (text-menu) and a full graphical version using JavaFX.

1.2 Motivation & Objectives

We selected an Employee Management System because it fits well with the fundamental objects concepts: one abstraction (Employee) and several implementations (Employee Category), data is encapsulated with validation (Employee Data), and the separation of data model, persistence layer, and user interface. It also allowed us to model a real institutional need rather than an artificial one, specifically AUW's registrar and HR.

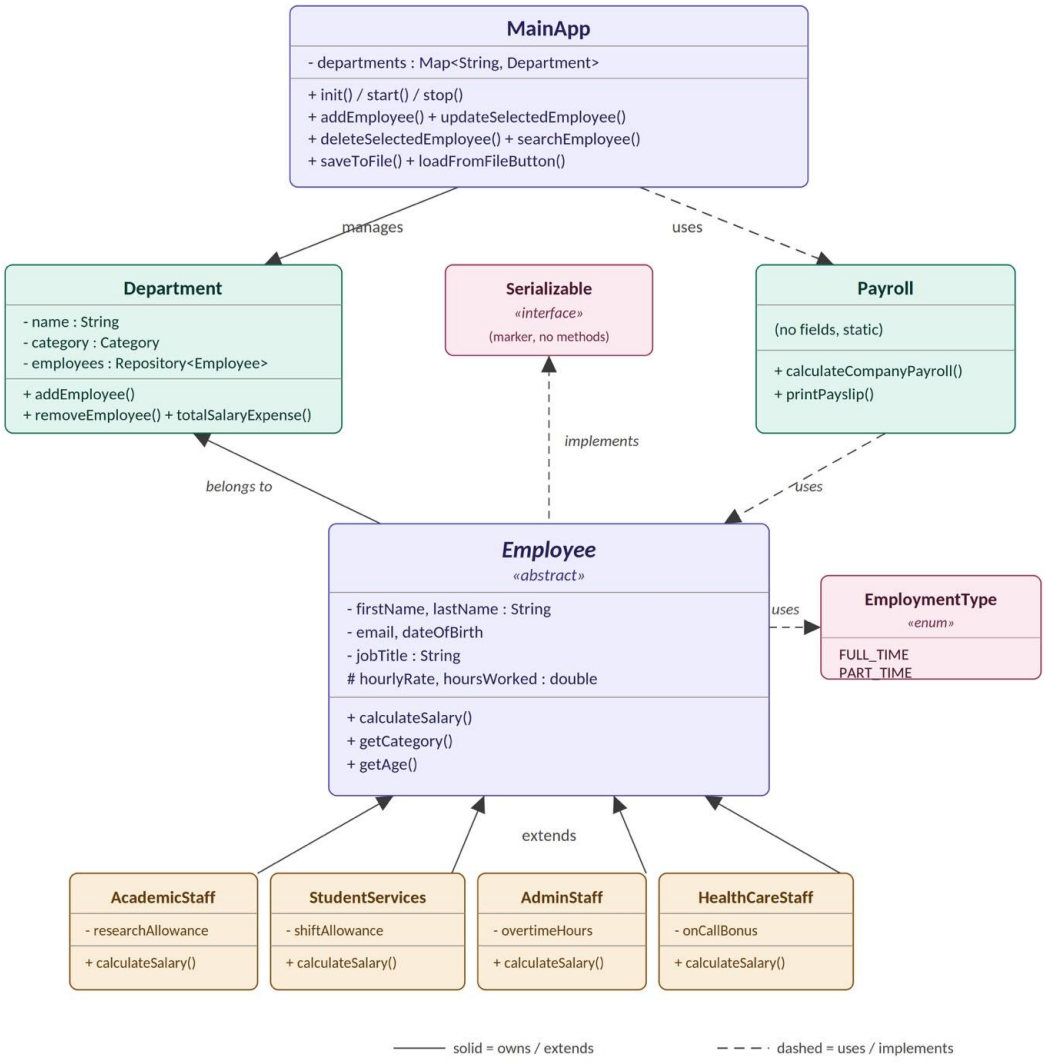
Project objectives:

1. *Create a class hierarchy based on inheritance and polymorphism, an interface called Employee and four classes: AcademicStaff, StudentServicesStaff, AdminStaff and HealthCareStaff that implement the calculateSalary () method using different salary policies.*
2. *Implement data persistence by using file I/O, saving and reloading all the departments and employees to and from a plain text file (employees.txt) between runs.*
3. *Develop a graphical interface for the domain model which is identical to that used for the console application, with the addition of a data table, Add/Update/Delete forms, a payroll summary, a bar-chart statistics view, and printing support, all built using JavaFX.*
4. *Encapsulate and provide custom exception handling across: It is appropriate to have private fields with valid guards on setters; It is appropriate to have a checked EmployeeNotFoundException thrown on failed lookups.*

2. System Analysis and Design

2.1 UML Diagram(s)

The class diagram below shows the core structure of the system: the abstract Employee superclass at the top, its four concrete subclasses (each adding one category-specific pay field and overriding calculateSalary()), the Department class that owns a generic Repository<Employee>, and the supporting Payroll and EmployeeNotFoundException classes.



2.2 Class Description

Class	Responsibility / Description
Employee (abstract)	*Base class for every staff member. Holds identity and employment fields (id, name, email, date of birth, job title, employment type, hourly rate, hours worked) and defines the abstract calculateSalary() and getCategory() that every subclass must implement.*
AcademicStaff	*Employee subclass for academic departments (Undergraduate, Graduate, Pathway, School, Fellows). Adds a research/teaching allowance expressed as a percentage of base pay.*
StudentServicesStaff	*Employee subclass for student-facing departments (Registration, Library, Residential Life, Visa Office, Work-Study). Adds a fixed shift allowance.*
AdminStaff	*Employee subclass for administrative departments (Finance, IT, Housekeeping, Maintenance, Security). Adds overtime hours paid at 1.5× the hourly rate.*
HealthCareStaff	*Employee subclass for health-care departments (Health Center, Mental Health). Adds a fixed on-call/clinical bonus.*
Department	Groups employees that belong to one Category. Delegates storage to a Repository<Employee>, exposes lookup by ID (throwing EmployeeNotFoundException on failure), and totals its own salary expense.
Repository<T>	A small generic, reusable collection wrapper around an ArrayList<T>, used by Department to store its employees. Demonstrates generics: the same class works for any element type.
Payroll	*Stateless utility class for cross-department salary math. Sums totalSalaryExpense() across all departments and formats a console payslip. Never recomputes pay itself — it always delegates to Employee.calculateSalary().*
EmployeeNotFoundException	*A custom checked exception thrown by Department.findEmployeeById() when no employee with the given ID exists in that department.*

Main (SystemController console)	Entry point for the text-menu version. Owns the Map<String, Department> of all departments, drives the menu loop, and reads/writes employees.txt.
MainApp (SystemController — GUI)	*JavaFX Application subclass and entry point for the graphical version. Builds the Home page and management screen, wires up the table, forms, dialogs, the statistics bar chart and printing, and auto-saves every change to the same employees.txt file the console version uses.*

2.3 System Workflow (Optional)

Both versions of the system have the same fluid flow, and the same model classes:

1. *The system starts with a fixed number of default departments (5 Academic, 5 Student Services, 5 Administration, 2 Health Care) and, in the JavaFX version, with a default set of ~15 employees the first time that it is started without any data.*
2. *The user selects a department and either adds a new employee (choosing the category-specific extra field — research allowance %, shift allowance, overtime hours, or on-call bonus) or looks up an existing one by ID or name.*
3. *Whenever an employee record is needed (for the table, a payslip, or a payroll total), the system calls the polymorphic calculateSalary() on the Employee reference; the correct subclass implementation runs automatically depending on the employee's actual runtime type.*
4. *Updates and deletions go through validated setters (invalid hourly rate or hours throw IllegalArgumentException) and are immediately reflected in payroll totals.*
5. *Every add, update, or delete triggers a save to employees.txt (a comma-separated line per employee, prefixed with its department). On the next launch, this file is parsed back into the same Department/Employee object graph.*

3. Implementation

3.1 Tools and Technologies

Category	Detail
Programming Language	*Java*
IDE	*Eclipse (project ships with .project / .classpath for Eclipse import)*
JDK Version	*JDK 26*
Libraries / Frameworks	*JavaFX (controls, charts, printing) for the GUI version; java.time for date handling; java.io for file persistence. No third-party libraries.*
GitHub Repository Link	*https://github.com/MarwaMossini/JavaFX_Project*

External Sources / References Used

1. *Oracle Java Tutorials — "Abstract Methods and Classes," used as a reference for the abstract Employee design.*
2. *Official JavaFX documentation (openjfx.io) — consulted for TableView, DatePicker, BarChart, and PrinterJob usage*.

3.2 OOP Concepts Applied

OOP Concept	Where and How It Is Used
Encapsulation	All fields in Employee are private (protected only for hourlyRate/hoursWorked, used by subclasses' salary math) with getters and validating setters — e.g. setHourlyRate() and setHoursWorked() throw IllegalArgumentException on negative values.
Inheritance	The abstract Employee class is extended by AcademicStaff, StudentServicesStaff, AdminStaff, and HealthCareStaff (all defined in Roles.java), each adding one category-specific field via its constructor.
Polymorphism	calculateSalary() and getCategory() are overridden differently in each subclass. Department.listEmployees() and Payroll.calculateCompanyPayroll() call these through Employee references, so the correct subclass logic runs at runtime without any type-checking code.
Abstraction	Employee is declared abstract and defines calculateSalary() and getCategory() as abstract methods, forcing every concrete role to specify its own pay rule and category while sharing all common employee data and behavior.
Generics	*Repository<T> is a single reusable, type-safe collection class; Department uses it as Repository<Employee>, but the same class could store any type.*

Custom Exceptions	*EmployeeNotFoundException (checked) is thrown by Department.findEmployeeById() and must be handled by every caller, e.g. in the payslip and update-employee flows.*
Interface	*The abstract Employee class implements Serializable (see 3.3). This is used as a contract that signals the class is designed to be persisted, without dictating the mechanism — records are actually written out as plain comma-separated text via toFileString(), not Java's binary serialization.*

3.3 Important Implementation Details

Hourly pay model with category specific bonuses

Every employee's pay starts from Employee.baseRegularPay() (hourlyRate × hoursWorked), defined once in the abstract base class so the core formula is never duplicated. Each subclass then overrides calculateSalary() to add its own bonus on top:

```
public abstract class Employee implements Serializable {
   protected double hourlyRate;
   protected double hoursWorked;
   protected double baseRegularPay() {
      return hourlyRate * hoursWorked;
   }
   public abstract double calculateSalary();
}
```

```
class AdminStaff extends Employee {
   private static final double OVERTIME_MULTIPLIER = 1.5;
   private double overtimeHours;
   @Override
   public double calculateSalary() {
      double overtimePay = overtimeHours * hourlyRate * OVERTIME_MULTIPLIER;
      return baseRegularPay() + overtimePay;
   }
}
```

Encapsulation with validation

Setters on Employee reject invalid input immediately rather than allowing bad data to reach the salary calculation:

```
public void setHourlyRate(double hourlyRate) {
   if (hourlyRate < 0) {
      throw new IllegalArgumentException("Hourly rate cannot be negative.");
   }
   this.hourlyRate = hourlyRate;
}
```

Department.findEmployeeById() throws EmployeeNotFoundException when no match exists, forcing every caller (payslip generation, employee updates) to explicitly handle a missing ID rather than letting a null reference propagate silently:

```
public Employee findEmployeeById(int id) throws EmployeeNotFoundException {
    for (Employee e : employeeRepo.getAll()) {
        if (e.getId() == id) return e;
    }
    throw new EmployeeNotFoundException(
        "Employee with ID " + id + " not found in department '" + name + "'.");
}
```

File persistence

Each Employee serializes itself to a comma-separated line via toFileString(); the base class writes the shared fields and each subclass appends its own extra field (e.g. AcademicStaff appends researchAllowancePercent). On load, Main / MainApp read the type name from the first column and reconstruct the correct subclass, rebuilding the full Department → Employee object graph from a single flat text file.

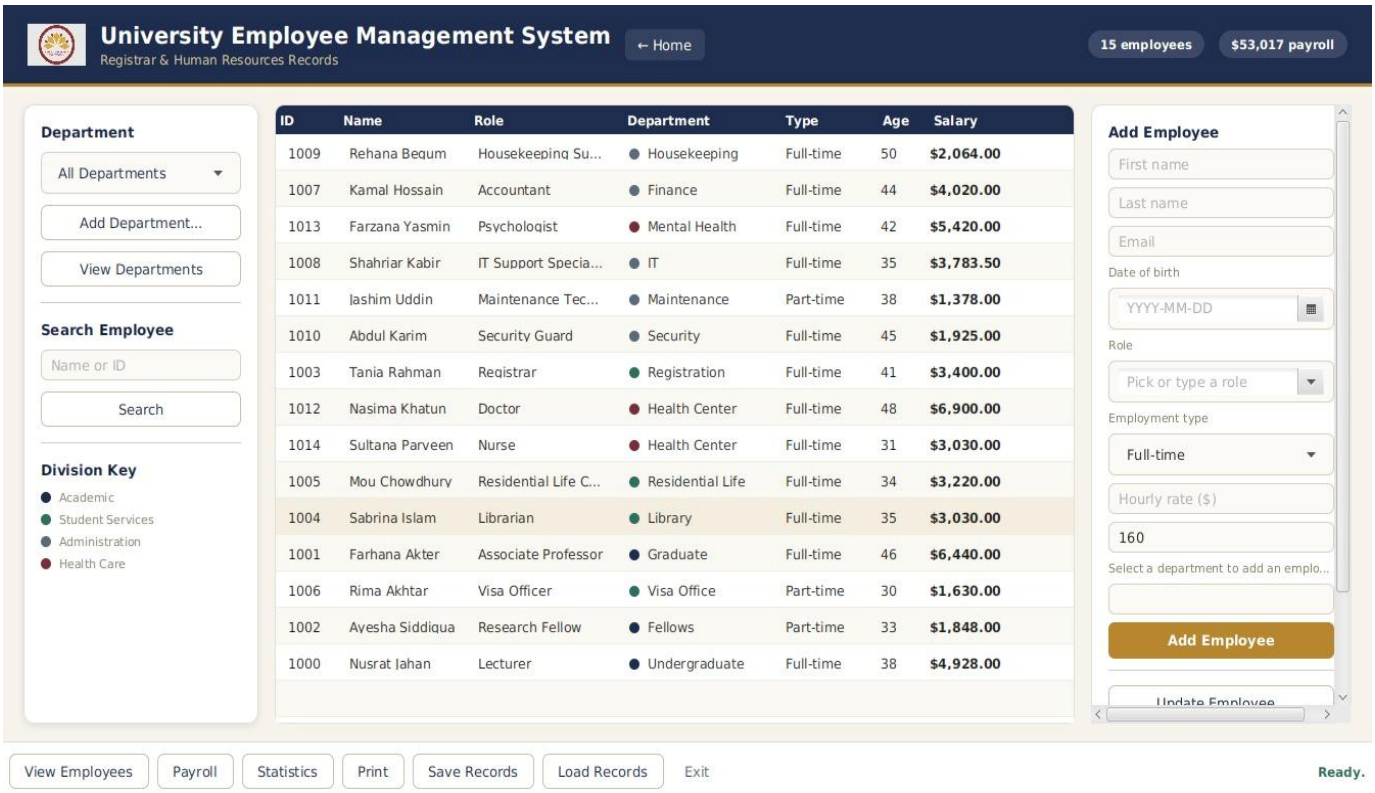
4. Testing and Results

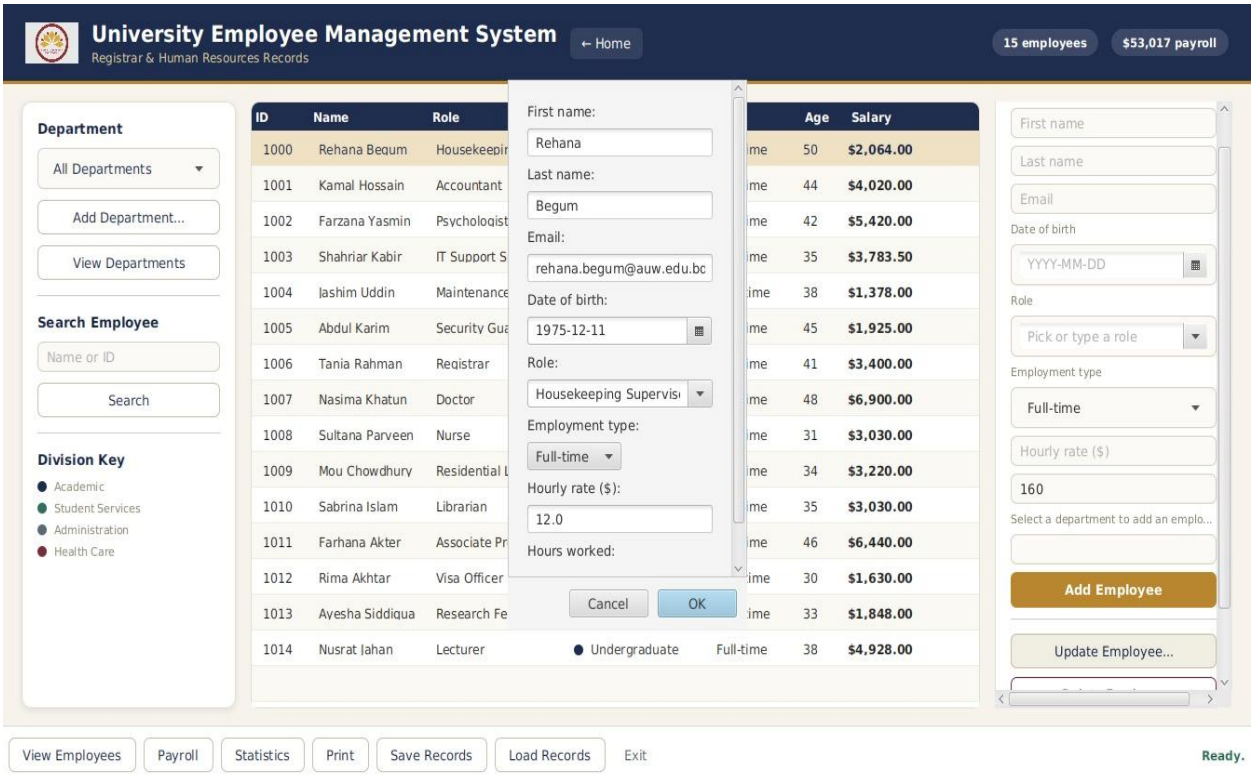
The system was tested manually by running both the console (Main) and JavaFX (MainApp) entry points and exercising each menu option / screen. Representative test cases:

Test Case	Sample Input	Expected Output	Actual Output
Add employee with "All Departments" selected	*Department dropdown left on "All Departments" while filling the Add Employee form*	*Add is blocked with a specific instruction to pick a real department; not a silent no-op*	*Matched expected — blocked with a clear instruction*
Add employee with negative hourly rate	*Hourly rate: -15*	*IllegalArgumentException("Hourly rate cannot be negative.") caught and shown as an error message; employee not added*	*Hourly rate cannot be negative.*
Add employee with non-numeric hourly rate	*Hourly Rate field: "abc"*	*NumberFormatException caught in a try-catch block and shown as an error message; employee not added*	*Matched expected — caught with a clear error message, no crash*
Update employee — all fields	*Open Update dialog for an existing employee; edit every field, including Date of Birth*	*All fields, including Date of Birth, are editable and the updated values are saved and displayed*	*Matched expected — every field including Date of Birth updated successfully*

Screenshots of Output / Application Interface






University Employee Management System
Registrar & Human Resources Records
← Home
15 employees
$53,017 payroll
Department
All Departments
Add Department...
View Departments
Search Employee
Name or ID
Search
Division Key
Academic
Student Services
Administration
Health Care
ID
Name
Role
Age
Salary
1000 Rehana Begum Housekeepir 50 $2,064.00
1001 Kamal Hossain Accountant 44 $4,020.00
1002 Farzana Yasmin Psychologist 42 $5,420.00
1003 Shahriar Kabir IT Support S 35 $3,783.50
1004 Jashim Uddin Maintenance 38 $1,378.00
1005 Abdul Karim Security Gua 45 $1,925.00
1006 Tania Rahman Registrar 41 $3,400.00
1007 Nasima Khatun Doctor 48 $6,900.00
1008 Sultana Parveen Nurse 31 $3,030.00
1009 Mou Chowdhury Residential L 34 $3,220.00
1010 Sabrina Islam Librarian 35 $3,030.00
1011 Farhana Akter Associate Pr 46 $6,440.00
1012 Rima Akhtar Visa Officer 30 $1,630.00
1013 Ayesha Siddiqua Research Fe 33 $1,848.00
1014 Nusrat Jahan Lecturer Undergraduate Full-time 38 $4,928.00
First name:
Rehana
Last name:
Begum
Email:
rehana.begum@auw.edu.bc
Date of birth:
1975-12-11
Role:
Housekeeping Supervisi
Employment type:
Full-time
Hourly rate ($):
12.0
Hours worked:
Cancel
OK
First name
Last name
Email
Date of birth
YYYY-MM-DD
Role
Pick or type a role
Employment type
Full-time
Hourly rate ($)
160
Select a department to add an emplo...
Add Employee
Update Employee...
View Employees
Payroll
Statistics
Print
Save Records
Load Records
Exit
Ready.

5. Challenges and Solutions

Some problems that need to be addressed:

Compatibility of versions of the console and of JavaFX:

Since there are two entry points, one for the console menu, and one for the GUI (Main and MainApp respectively), a risk early on in this project is that they may diverge in how they handle the Employee / Department / Payroll model, for instance a bug fix in one but not the other. We did this by storing all business logic (salary rules, validation, file format) in the shared model classes, and Main and MainApp do not share any calculation logic, just the input/output layer.

JavaFX DatePicker silently losing typed dates:

One of the common issues is that JavaFX DatePicker silently loses the typed date. We noticed that JavaFX's DatePicker does not always perform the commit of the typed date when tabbed out or when it loses focus, depending on the JVM's default locale matching the typed date format; this resulted in a silent null date Of Birth that was reported downstream as an age of -1. We did this in two ways: First, we had an explicit ISO yyyy-MM-dd String Converter on the picker and, second, we had a resolveDatePickerValue() fall-back that would read the raw text back off the editor if the picker returned null. The addEmployee() method was also modified to not allow saving without an employee's date, and so this failure no longer occurs silently.

Modeling four different pay structures without duplicating code:

Used a base class for a base pay, and then created derived classes for each of the four pay structures that are going to be used. This was an attractive idea, to pass one calculateSalary() to Employee, and have an if/else branch condition on category. This is the polymorphism that the assignment requested us to show: We've implemented it as an abstract method, with one override per subclass, such that adding or modifying a category's pay rule never can affect the other categories, and Department/Payroll never will need to know which concrete subclass they're working with. The problem is to reload the subclasses of typed objects from a flat text file.

Reloading typed subclasses from a flat text file:

This means that when loading employees.txt back in as the corresponding subclass, it would be necessary to store a simple class name on the first field of each line and switch on that class during load time, and then use the parsing to determine the trailing category-specific column to load. A source of bugs in early days was the malformed field or the missing field in extra fields, a problem solved by making it very clear that toFileString() and the column order in the loader is strictly maintained in all four subclasses.

6. Conclusion

What was achieved:

The team designed and implemented the full University Employee Management System (UEM) with the console application and the full JavaFX GUI, both having the same domain model. The following three objectives were achieved: an inheritance hierarchy of four Employee subclasses, all department and employee data stored in files, and a JavaFX interface which includes a data table, workflows for adding, updating and deleting employee and department data, a payroll summary, a statistics bar chart and printing support.

OOP concepts learned/applied:

Abstraction and Polymorphism was most evident in the abstract Employee class; the same call to calculateSalary() was executed in four different ways depending on the runtime type of the employee, making it clear why it is better to override rather than use conditional logic. Validating setters and a custom checked exception (EmployeeNotFoundException) made the point of failing loudly and early even more compelling! One of the most new ideas in the team was Generics (Repository<T>), those were the most time consuming to make the type parameters correct.

Future improvements:

- *Use a real database (e.g. SQLite) instead of a flat-file persistence system for concurrent access and more advanced query support.*
- *Add authentication/roles to ensure that only HR staff can edit the salary-affecting fields.*
- *Include input-level unit tests (JUnit) for each subclass's calculateSalary() formula instead of just testing it manually.*
- *Add payroll by division and year over year comparisons to the Statistics view.*